

Risultati Sperimentali Modal Dropout su DETR

Le seguenti tabelle riportano i risultati ottenuti sotto diverse configurazioni di *modal dropout* applicate al modello DETR. Durante la fase di valutazione, la modalità di input viene campionata in base a probabilità predefinite $P(\text{IR})$, $P(\text{RGB})$ e $P(\text{Fusion})$.

Addestramento su $D_{\text{RGB-IR}}$

Table 1: Risultati del modello addestrato su $D_{\text{RGB-IR}}$ con diverse configurazioni di modal dropout.

Testato su	$P(\text{IR})$	$P(\text{RGB})$	$P(\text{Fusion})$	mAP@50
D_{IRo}	100	0	0	0.138
D_{IRo}	90	0	10	0.132
D_{IRo}	50	50	0	0.163
D_{IRo}	37.5	37.5	25	0.132
D_{IRo}	25	25	50	0.159
D_{IRo}	12.5	12.5	75	0.000
D_{IRo}	20	20	60	0.173
$D_{\text{RGB-IR}}$	100	0	0	0.379
$D_{\text{RGB-IR}}$	90	0	10	0.300
$D_{\text{RGB-IR}}$	50	50	0	0.388
$D_{\text{RGB-IR}}$	37.5	37.5	25	0.302
$D_{\text{RGB-IR}}$	25	25	50	0.377
$D_{\text{RGB-IR}}$	12.5	12.5	75	0.000
$D_{\text{RGB-IR}}$	20	20	60	0.417

Addestramento su $D_{\text{RGBs-IRa}}$

Table 2: Risultati del modello addestrato su $D_{\text{RGBs-IRa}}$ con diverse configurazioni di modal dropout.

Testato su	$P(\text{IR})$	$P(\text{RGB})$	$P(\text{Fusion})$	mAP@50
D_{IRo}	100	0	0	0.224
D_{IRo}	90	0	10	0.261
D_{IRo}	50	50	0	0.227
D_{IRo}	37.5	37.5	25	0.306
D_{IRo}	25	25	50	0.247
D_{IRo}	12.5	12.5	75	0.196
D_{IRo}	20	20	60	0.200
$D_{\text{RGBs-IRa}}$	100	0	0	0.224
$D_{\text{RGBs-IRa}}$	90	0	10	0.252
$D_{\text{RGBs-IRa}}$	50	50	0	0.219
$D_{\text{RGBs-IRa}}$	37.5	37.5	25	0.280
$D_{\text{RGBs-IRa}}$	25	25	50	0.233
$D_{\text{RGBs-IRa}}$	12.5	12.5	75	0.205
$D_{\text{RGBs-IRa}}$	20	20	60	0.220

Addestramento su D

Table 3: Risultati del modello addestrato su D con diverse configurazioni di modal dropout.

Testato su	$P(\text{IR})$	$P(\text{RGB})$	$P(\text{Fusion})$	mAP@50
D_{IRo}	100	0	0	0.270
D_{IRo}	90	0	10	0.320
D_{IRo}	50	50	0	0.275
D_{IRo}	37.5	37.5	25	0.293
D_{IRo}	25	25	50	0.270
D_{IRo}	12.5	12.5	75	0.158
D_{IRo}	20	20	60	0.256
D	100	0	0	0.322
D	90	0	10	0.361
D	50	50	0	0.331
D	37.5	37.5	25	0.333
D	25	25	50	0.325
D	12.5	12.5	75	0.286
D	20	20	60	0.323

Table 4: Confronto diretto tra i risultati originali e i migliori risultati ottenuti con Modal Dropout. Le righe sono raggruppate per set di addestramento e test per evidenziare l’impatto della nuova metodologia.

Trainato su	Testato su	$P(\text{IR})$	$P(\text{RGB})$	$P(\text{Fusion})$	mAP@50
D	D	0	0	100	0.372
	D	90	0	10	0.361
	D_{IRo}	0	0	100	0.233
	D_{IRo}	90	0	10	0.320
$D_{\text{RGB-IR}}$	D_{IRo}	0	0	100	0.000
	D_{IRo}	20	20	60	0.173
$D_{\text{RGBs-IRa}}$	D_{IRo}	0	0	100	0.000
	D_{IRo}	37.5	37.5	25	0.306

Evoluzione con RT-DETR (Real-Time DETR)

In questa fase della ricerca, l'obiettivo è stato adattare la logica multimodale su un'architettura **RT-DETR**, ottimizzata per l'inferenza real-time. Sono state testate due strategie principali per gestire la natura multi-scala del modello e la risoluzione ridotta dei target.

RT-DETR con Additive Feature Fusion

Questa architettura utilizza una **doppia backbone** (ResNet-50vd) per elaborare separatamente le modalità RGB e IR. Le *feature maps* multi-scala estratte ($P3, P4, P5$) vengono fuse tramite **somma elemento per elemento** (Additive Fusion) e successivamente processate dall'*Hybrid Encoder* (AIFI e CCFM).

Questa scelta presenta due vantaggi chiave:

1. preserva l'allineamento spaziale tra le modalità, mantenendo la compatibilità con il meccanismo di *Deformable Attention* del decoder;
2. non introduce parametri addizionali, evitando instabilità durante il *fine-tuning* del modello pre-addestrato su ImageNet.

Il modello è stato addestrato per 10 e 25 epoche utilizzando una strategia di *modal dropout* (20% IR-only, 20% RGB-only, 60% fusion - rivelatasi la migliore durante i test sul modello DETR) per migliorare la robustezza delle modalità singole.

Table 5: Risultati RT-DETR (Fusion) trainato per 25 epoche su immagine integrale al variare della soglia di confidenza.

Testato su	Threshold	mAP@50 (RT-DETR)	mAP@50 (DETR)
$D_{\text{RGB-IR}}$	0.01	0.321	
$D_{\text{RGB-IR}}$	0.05	0.287	0.417
$D_{\text{RGB-IR}}$	0.10	0.246	
D_{IRo}	0.01	0.262	0.173
D_{VISo}	0.01	0.264	-

Table 6: Risultati RT-DETR (Fusion) trainato per 10 epoche su immagine integrale al variare della soglia di confidenza.

Testato su	Threshold	mAP@50 (RT-DETR)	mAP@50 (DETR)
$D_{\text{RGB-IR}}$	0.01	0.357	
$D_{\text{RGB-IR}}$	0.05	0.357	0.417
$D_{\text{RGB-IR}}$	0.10	0.356	
D_{IRo}	0.01	0.252	0.173
D_{VISo}	0.01	0.246	-

Da queste tabelle notiamo come il training per 10 epoche abbia dato risultati migliori sul dataset fusion ma peggiori sulle singole modalità (RGB/IR). Un aspetto interessante è che in questo caso anche la confidence del modello sembra essere maggiore visto che non c'è quasi alcun peggioramento di prestazioni tra il test con il threshold più basso (0.01) ed il più alto (0.1). Inoltre, rispetto a DETR questo nuovo approccio peggiora le prestazioni sul dataset fusion (0.417 vs 0.357) ma migliora notevolmente le prestazioni sulle singole modalità ($\text{IR} \approx 0.25$ vs ≈ 0.17).

RT-DETR con Strategia di Tiling (SAHI-style)

Per ovviare alla dimensione ridotta dei soggetti, è stato testato un approccio basato su **tiling**: l'immagine 640×640 viene divisa in quadranti 320×320 , raddoppiando la scala relativa degli oggetti. Durante il test, le predizioni sui singoli tile vengono rimappate e aggregate tramite NMS globale.

Durante l'**addestramento**, durato anche in questo caso 10 e 25 epoche, ogni immagine viene ritagliata in modo casuale su uno dei 4 quadranti, mentre durante il **test** tutti i quadranti vengono processati e aggregati tramite NMS globale (soglia IoU 0.5).

Table 7: Confronto della robustezza multimodale: RT-DETR Integrale vs RT-DETR con Tiling.

Test Dataset	Modello Integrale (mAP@50)	Modello Tiling (25 ep.) (mAP@50)	Modello Tiling (10 ep.) (mAP@50)
$D_{\text{RGB-IR}}$	0.321	0.302	0.151
D_{IRo}	0.262	0.133	0.105
D_{VISo}	0.264	0.082	0.050

*I risultati del modello Tiling si riferiscono all'inferenza con aggregazione dei quadranti.

Tutti i valori sono riportati con soglia di confidenza 0.01 per evidenziare il potenziale di localizzazione.

RT-DETR con Framework CMX (SOTA Fusion)

È stata infine testata l'integrazione del framework **CMX** (Zhang et al. 2023), che introduce due moduli avanzati: il *Cross-Modal Feature Rectification Module* (CM-FRM) per calibrare i sensori e il *Feature Fusion Module* (FFM) basato su *Cross-Attention*. L'obiettivo era verificare se una fusione adattiva potesse gestire meglio i disallineamenti spaziali del dataset.

Il modello è stato addestrato per 25 epoche. I risultati evidenziano una netta degradazione delle prestazioni rispetto alla somma additiva.

Table 8: Risultati RT-DETR con integrazione CMX (25 epoche).

Testato su	Threshold	mAP@50 (CMX)	mAP@50 (Base)
$D_{\text{RGB-IR}}$	0.01	0.032	0.321
$D_{\text{RGB-IR}}$	0.10	0.000	0.246
D_{IRo}	0.01	0.081	0.262
D_{VISo}	0.01	0.018	0.264

Questi risultati evidenziano come l'elevata complessità dei moduli CMX (molteplici parametri aggiuntivi in MLP e Attention) introduca un'eccessiva inerzia nel processo di ottimizzazione su un dataset di dimensioni ridotte come il WiSARD. Si osserva un fenomeno di **interferenza distruttiva**: la modalità *Fusion* (0.032) risulta notevolmente inferiore alla modalità *IR-only* (0.082), indicando che il modello non è riuscito a isolare il segnale termico dal rumore visivo. Inoltre, la confidenza del modello è rimasta estremamente bassa, portando a prestazioni nulle con soglia 0.1.

Il Tentativo Ibrido: FAM + CMX + Positional Encoding

Dall'analisi dei fallimenti del modello CMX puro, sono emerse tre criticità principali strutturali legate allo specifico dominio SAR (Search And Rescue):

1. **Disallineamento e Parallasse:** Il modulo di rettifica (CM-FRM) di CMX assume che le feature map siano "pixel-perfect". In presenza di disallineamento, usare il valore infrarosso

per calibrare il pixel visivo sottostante significa mescolare il target con il rumore di fondo, causando una grave interferenza distruttiva.

2. **Perdita di Contesto Spaziale nel FFM:** Il modulo di fusione basato su *Cross-Attention* srotola i token temporaneamente privandoli della cognizione 2D. Senza le posizioni relative, si genera *Ghosting* (l'RGB preleva informazioni termiche da parti casuali ma semanticamente affini o "calde" dell'immagine).
3. **Collo di bottiglia su P3:** A causa della complessità quadratica $O(N^2)$ dell'attenzione, al livello P3 (quello a più alta risoluzione, vitale per trovare bersagli minuscoli) è necessario applicare un fallback lineare (bypass). Fondendo mappe disallineate con una semplice concatenazione lineare si distruggono irrimediabilmente i piccoli target.

Per superare e risolvere teoricamente queste defezioni è stata progettata un'Innovativa **Architettura Ibrida** che incapsula i vantaggi di entrambi gli approcci esplorati. La pipeline *Triple-Step* elaborava le feature nel seguente ordine: $[RGB, IR] \rightarrow FAM \rightarrow CM - FRM \rightarrow FFM$.

In pratica, prima si utilizzavano le Deformable Convolutions del FAM per allineare l'IR ed eliminare la parallasse; poi si procedeva con la calibrazione incrociata (ora corretta spazialmente); infine, le feature venivano fuse nel modulo FFM a cui però è stato iniettato un **2D Sine Positional Encoding** customizzato per prevenire il ghosting dell'attenzione.

Il modello è stato addestrato e di seguito sono riportati i risultati finali:

Table 9: Risultati RT-DETR Architettura Ibrida (FAM + CMX + Pos. Encoding).

Testato su	Threshold	mAP@50 (CMX Ibrido)	mAP@50 (CMX Puro)
D_{RGB-IR}	0.01	0.1447	0.032
D_{RGB-IR}	0.10	0.0878	0.000
D_{IRo}	0.01	0.0944	0.081
D_{IRo}	0.10	0.0543	0.000
D_{VISo}	0.01	0.1023	0.018
D_{VISo}	0.10	0.0607	0.000

I risultati dimostrano che le intuizioni teoriche erano corrette: l'aggiunta dell'allineamento spaziale e posizionale ha risollevato l'architettura dal collasso totale, riportando le prestazioni multimodali da uno 0.032 a un 0.1447 (e rendendo la rete finalmente in grado di fare previsioni fiduciose anche a iterazione stretta $Thr = 0.10$, dove il CMX originale falliva totalmente con 0.000). Tuttavia, confrontando i valori con la semplice *Additive Fusion* o con la nostra variante regolarizzata SSJ (0.438 mAP), è palese che la mole di parametri e la pesantezza strutturale del blocco CMX generano comunque *overfitting* e inerzia estrattiva. Questo esperimento certifica matematicamente che, su task RGBT limitati basati su droni, operazioni di allineamento euristiche e leggere surclassano pesanti framework multimodali attentivi standard.

1 Evoluzione con Deformable DETR: Dalla Fusione Spaziale alla Channel Concatenation

Successivamente la ricerca si è spostata sull'architettura **Deformable DETR**. L'obiettivo era sfruttare il meccanismo di attenzione sparsa (Deformable Attention) per gestire meglio le feature multi-scala, cercando un compromesso tra la complessità computazionale del CMX e la semplicità della somma additiva.

In questa fase sono stati esplorati due approcci distinti di fusione delle feature, portando allo sviluppo di un'architettura dedicata basata sulla concatenazione dei canali.

1.1 Il fallimento della Fusione Spaziale (Spatial Concatenation)

Il primo tentativo di adattamento ha previsto una strategia di *Spatial Concatenation*. In questa configurazione, le feature map provenienti dalla backbone RGB e da quella IR sono state concatenate lungo l'asse dell'altezza (dimensione H), raddoppiando di fatto la dimensione verticale del tensore di input per l'encoder ($H_{in} \rightarrow 2H$).

L'ipotesi era che il meccanismo di attenzione deformabile potesse apprendere autonomamente le relazioni tra le due modalità. Tuttavia, l'approccio ha presentato due criticità fatali:

- **Esplosione Computazionale:** Il raddoppio delle dimensioni spaziali ha costretto a ridurre i livelli delle feature map (da 4 a 2) per evitare problemi di memoria (OOM), limitando la capacità del modello di rilevare oggetti a diverse scale.
- **Rottura Semantica:** Alterare la geometria spaziale invalida i *reference points* pre-addestrati, introducendo una discontinuità che il modello non è riuscito a compensare, portando a metriche mAP prossime allo zero.

1.2 Implementazione della Channel Concatenation con Proiezione

Per preservare l'allineamento spaziale e la capacità rappresentativa originale, è stata implementata una nuova strategia basata sulla **Channel Concatenation**, definita nel modulo `DeformableDetrFusionBackbone`.

A differenza dell'approccio precedente, questa architettura mantiene inalterate le dimensioni spaziali ($H \times W$) e opera la fusione lungo la dimensione dei canali. Il processo si articola nei seguenti step:

1. **Dual Backbone Indipendente:** Vengono istanziate due backbone parallele (ResNet-50) per estrarre le feature separatamente da RGB e IR. I pesi della backbone IR sono inizializzati adattando quelli pre-addestrati su RGB (media dei canali) per favorire la convergenza.
2. **Concatenazione e Fusione Non Lineare:** Per ogni livello della piramide di feature, le feature F_{RGB} e F_{IR} vengono inizialmente concatenate lungo la dimensione dei canali ($2C_i$). Invece di una semplice proiezione lineare, viene applicato un blocco di fusione più strutturato composto da:
 - **Convoluzione 1x1:** Riduce i canali da $2C_i$ a C_i , operando una prima miscelazione delle informazioni.
 - **Group Normalization:** Stabilizza l'output (32 gruppi), garantendo robustezza statistica anche con *batch size* molto ridotti (`batch_size=1` nella configurazione adottata).
 - **Activation (ReLU):** Introduce non-linearità, permettendo al blocco di modulare il contributo di ciascuna modalità in funzione del segnale piuttosto che limitarsi a una combinazione lineare fissa.

$$F_{fused} = \text{ReLU}(\text{GN}(\text{Conv}_{1 \times 1}(\text{Concat}(F_{RGB}, F_{IR}))))$$

Questo blocco di fusione è condiviso, senza modifiche, da tutte le configurazioni discusse in questo capitolo (baseline, FAM, FAM + SSJ) e dall'architettura DINO descritta nel capitolo successivo.

3. **Mantenimento della Multi-Scala:** Grazie all'efficienza di questa fusione, è stato possibile mantenere attivi tutti i **4 feature levels** standard di Deformable DETR, garantendo la massima robustezza nel rilevamento di oggetti di piccole dimensioni senza incorrere in colli di bottiglia computazionali.

Le feature fuse $F_{fused,i}$ di ciascun livello i vengono ulteriormente proiettate a una dimensione comune $d_{model} = 256$ tramite una convoluzione 1×1 per livello, per poi essere appiattite (*flatten*) e concatenate in un'unica sequenza di token fornita all'encoder. Con un input 600×600 e backbone ResNet-50, i quattro livelli hanno risoluzione 75×75 , 38×38 , 19×19 e 10×10 , per un totale di $5625 + 1444 + 361 + 100 = 7530$ token da 256 dimensioni ciascuno (Figura 1).

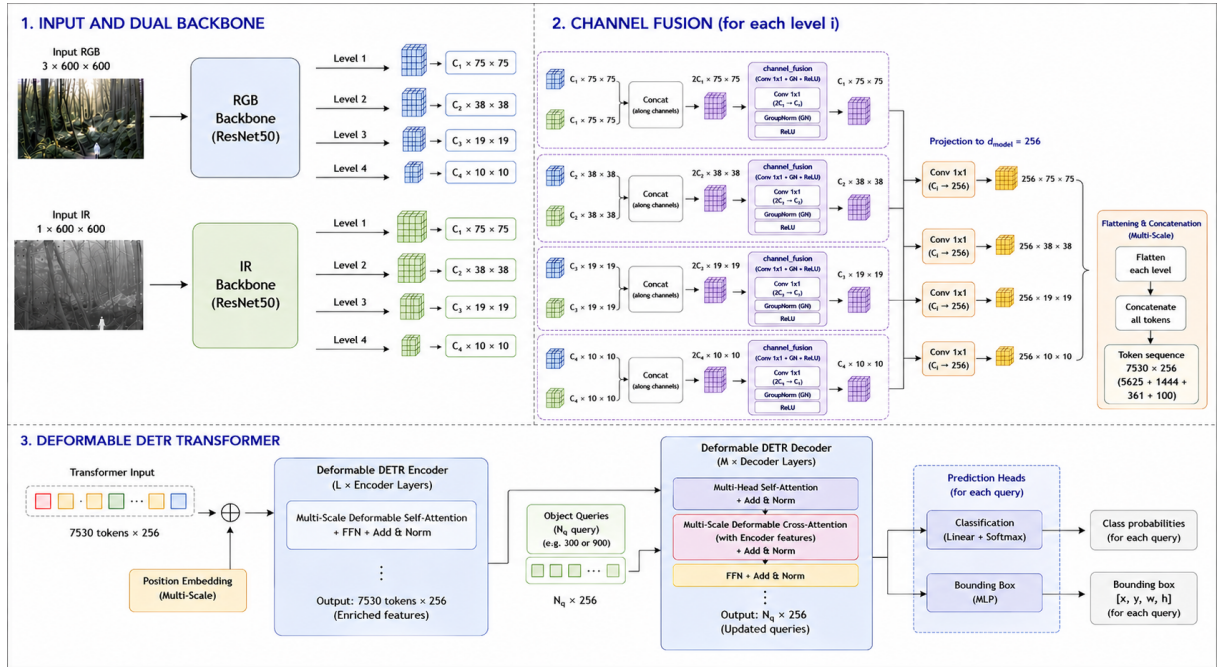


Figure 1: Architettura proposta di Deformable DETR Fusion. Vengono mostrati il flusso dalla Dual Backbone alla Channel Fusion non lineare per ogni livello della piramide di feature, la proiezione a $d_{model} = 256$ e la generazione della sequenza di 7530 token per l'encoder, seguita dal transformer deformabile e dalle teste di predizione.

Le feature multiscale così ottenute vengono passate, insieme alle posizionali multi-scala, all'encoder di Deformable DETR (*self-attention* deformabile multi-scala), che produce una rappresentazione arricchita della stessa lunghezza (7530 token). Il decoder, dato un insieme di N_q query apprendibili (300 nella configurazione adottata), alterna self-attention standard tra le query e cross-attention deformabile multi-scala verso le feature dell'encoder; l'output finale di ciascuna query viene proiettato da due teste di predizione indipendenti — una di classificazione (lineare + softmax) e una di regressione del bounding box (MLP) — per produrre le predizioni finali.

Questa configurazione garantisce che ogni "pixel" nelle feature map in ingresso al Transformer contenga un'informazione ibrida (RGB+IR) perfettamente allineata, permettendo al modello di sfruttare i pesi pre-addestrati di Deformable DETR senza subire disallineamenti geometrici.

1.3 Integrazione del modulo FAM e SSJ

A seguito dei risultati positivi ottenuti su RT-DETR, lo stesso schema di allineamento è stato integrato anche nella backbone di fusione di Deformable DETR, tramite il `FeatureAlignmentModule` già impiegato per RT-DETR. Per ciascun livello della piramide, le feature RGB e IR vengono concatenate e passate a una `Conv2d (offset_conv)` che produce 27 canali: 18 codificano le coordinate (x, y) di 9 punti di campionamento deformabili, i restanti 9 i corrispondenti pesi di modulazione (*mask*). Tali offset vengono applicati esclusivamente alla feature map IR tramite una `DeformConv2d (DCNv2)`, che la riallinea spazialmente rispetto alla feature RGB prima della concatenazione e proiezione canale descritta in precedenza; i pesi di `offset_conv` sono inizializzati a zero, per cui all’inizio del training il modulo si comporta come un’identità e l’allineamento viene appreso gradualmente durante l’addestramento.

Lo **Stochastic Spatial Jitter (SSJ)** aggiunge, esclusivamente durante il training, rumore gaussiano (deviazione standard `spatial_jitter_std`) direttamente sugli offset predetti, prima del passaggio alla `DeformConv2d`. L’effetto è impedire al modulo FAM di convergere verso un singolo pattern di offset fisso — potenzialmente una scorciatoia spuria specifica al training set — forzandolo invece ad apprendere un campo di allineamento robusto a piccole perturbazioni: lo stesso principio di regolarizzazione già validato su RT-DETR, qui trasportato sull’architettura Deformable DETR.

1.4 Instabilità Run-to-Run dell’Attenzione Deformabile

Durante la campagna sperimentale su questa famiglia di modelli è emerso un fenomeno non osservato con RT-DETR: a parità di seed, esecuzioni successive del medesimo addestramento producono risultati finali significativamente diversi (variazioni di mAP@50 anche superiori a 0.10). Poiché ogni sorgente di stocasticità controllata dal seed (ordine di shuffling del DataLoader, scelta della modalità in `modal_dropout`) è deterministica a parità di seed, questa osservazione esclude di per sé tali fattori come causa: la sorgente di non-determinismo risiede altrove.

La causa è stata identificata nel kernel CUDA che implementa la *Multi-Scale Deformable Attention*. Il passo all’indietro (*backward*) di questo kernel deve retro-propagare il gradiente verso le posizioni campionate (non intere) della feature map; poiché più query possono campionare la stessa cella o celle adiacenti, l’accumulo richiede operazioni `atomicAdd`. L’ordine con cui i thread eseguono questi accumuli non è deterministico, e poiché la somma in virgola mobile non è associativa, l’esito numerico del backward cambia leggermente a ogni esecuzione, anche a parità di seed e di dato in ingresso. RT-DETR, basato su attenzione standard *scaled dot-product*, ha un backward interamente deterministico: questo spiega perché la stessa instabilità non sia mai stata osservata in quell’architettura.

Una piccola perturbazione numerica nel gradiente non spiegherebbe, da sola, oscillazioni di 0.10 mAP: l’amplificazione avviene tramite il matcher di Hungarian (`DeformableDetrHungarianMatcher`), che assegna in modo discreto ogni query alle ground truth disponibili. Tale assegnazione è una funzione discontinua dei logit e delle box predette: una minima divergenza numerica può alterare l’assegnazione già nelle prime iterazioni, portando le query a specializzarsi su oggetti diversi in esecuzioni diverse e le rispettive traiettorie di addestramento a divergere in modo permanente.

Si è inoltre verificato che `torch.use_deterministic_algorithms(True)` non costituisce una soluzione praticabile: il kernel di Deformable Attention non possiede un’implementazione CUDA deterministica, per cui l’attivazione del flag forza un fallback estremamente lento (epoche da oltre 4 ore contro i ~35–40 minuti tipici), rendendo l’addestramento deterministico non sostenibile su questo dataset.

Di conseguenza, per ogni configurazione basata su Deformable DETR (baseline, FAM, FAM + SSJ, DINO) sono state eseguite 5 run con seed differenti (43, 44, 45, 46 e, a seconda della

configurazione, 47 oppure 49)¹, e i risultati sono riportati come **mediana e range** piuttosto che come media e deviazione standard, essendo la mediana più robusta a singole run anomale su campioni di numerosità così ridotta ($n = 5$).

Un’analisi di potenza statistica conferma che $n = 5$ non è sufficiente a distinguere le configurazioni testate: per confermare con potenza dell’80% ($\alpha = 0.05$) il guadagno medio osservato per il FAM (+0.023 mAP, deviazione standard aggregata ≈ 0.04) sarebbero necessarie circa 48 run per configurazione; utilizzando il guadagno sulla mediana (+0.045), la stima più favorevole si riduce a circa 13 run per configurazione. La conclusione onesta, a parità di risorse computazionali disponibili, è che le configurazioni testate sono **indistinguibili al numero di seed attuale**; tra di esse, il **FAM mostra la tendenza più promettente**, sia in termini di mediana che di sovrapposizione del range con la baseline.

1.5 Risultati su Deformable DETR: Baseline, FAM e FAM + SSJ

La Tabella 10 riporta mediana e range su 5 seed per le tre configurazioni testate, a soglia di confidenza 0.01.

Table 10: Risultati Deformable DETR (Fusion), confidence threshold 0.01. Per le configurazioni basate su Deformable DETR i valori sono mediana [range] su 5 run con seed differenti; per RT-DETR e DETR (attenzione standard, deterministica) si riportano i valori a singola run.

Configurazione	D_{VISo}	D_{IRo}	$D_{\text{RGB-IR}}$
Deformable DETR (baseline)	0.157 [0.131–0.197]	0.115 [0.086–0.142]	0.249 [0.199–0.303]
+ FAM	0.184 [0.171–0.253]	0.131 [0.097–0.165]	0.294 [0.213–0.314]
+ FAM + SSJ	0.142 [0.115–0.206]	0.088 [0.073–0.109]	0.263 [0.170–0.350]
RT-DETR (singola run)	0.246	0.252	0.357
DETR (singola run)	–	0.173	0.417

Il modulo FAM è l’unica variante che mostra una separazione visibile sulla mediana rispetto alla baseline (0.294 contro 0.249 in $D_{\text{RGB-IR}}$), sebbene il range superiore della baseline (0.303) si sovrapponga ampiamente al range del FAM. Lo Stochastic Spatial Jitter, che in RT-DETR aveva un effetto stabilizzante, produce qui l’effetto opposto: il range di FAM + SSJ (0.170–0.350) è il più ampio delle tre configurazioni, quasi il doppio di baseline e FAM. Una possibile spiegazione è che il rumore spaziale iniettato dallo SSJ interagisca con il non-determinismo del kernel di Deformable Attention descritto in §1.4, sommando due sorgenti di varianza indipendenti invece di compensarne una con l’altra come accade con l’attenzione standard di RT-DETR.

In nessun caso i tre intervalli risultano statisticamente separabili al numero di seed attuale; anche il confronto con il singolo valore disponibile per RT-DETR (0.357) o DETR (0.417) va quindi letto come indicativo, non come una differenza consolidata.

¹La run con seed 42 è stata esclusa da tutte le sweep defDETR/DINO a causa di un tempo per epoca anomalo ($\sim 2\text{h}$ vs $\sim 35\text{--}40\text{min}$).

2 Ottimizzazione di RT-DETR: Il Modulo FAM (Feature Alignment Module)

Nonostante l'efficienza di RT-DETR, l'additive fusion soffre dei disallineamenti spaziali tra i sensori RGB e Termici.

Per risolvere il problema geometrico senza appesantire il modello, è stato implementato un **Feature Alignment Module (FAM)** basato su convoluzioni deformabili. Questo modulo è inserito tra l'uscita delle backbone e la fusione additiva.

2.1 Architettura del FAM

L'idea è utilizzare la ricchezza semantica e la risoluzione superiore dell'immagine RGB per calcolare come "distorcere" la feature map termica (IR) affinché combaci perfettamente con quella RGB.

Il funzionamento, definito nella classe `FeatureAlignmentModule`, segue questi passaggi per ogni livello della piramide di feature:

1. **Offset Learning:** Le feature map RGB (F_{RGB}) e IR (F_{IR}) vengono concatenate lungo i canali. Una convoluzione standard 3×3 predice gli offset spaziali (Δp) e le maschere di modulazione (m) per ogni posizione del kernel:

$$[\Delta p, m] = \text{Conv}_{3 \times 3}(\text{Concat}(F_{RGB}, F_{IR}))$$

Il livello di output ha 27 canali (per un kernel 3×3): 18 per le coordinate (x, y) dei 9 punti di campionamento e 9 per gli scalari di modulazione (mask).

2. **Deformable Alignment:** Gli offset calcolati vengono applicati *esclusivamente* alla feature map IR tramite una Deformable Convolution (DCNv2):

$$F'_{IR}(p) = \sum_{k=1}^K w_k \cdot F_{IR}(p + p_k + \Delta p_k) \cdot m_k$$

dove p è la posizione spaziale, p_k l'offset fisso del kernel e Δp_k l'offset appreso.

3. **Fusione Additiva:** La feature map IR riallineata viene sommata a quella RGB originale:

$$F_{fused} = F_{RGB} + F'_{IR}$$

2.2 Dettagli Implementativi ed Evoluzione Architetture sul FAM

Lo sviluppo del FAM non è stato lineare, i risultati più interessanti sono emersi a partire da un bug nella prima implementazione. Di seguito descrivo le cinque versioni testate, cercando di spiegare il ragionamento dietro ogni scelta.

Fase 1: Il bug della Lazy Initialization (Modello A)

La prima versione del FAM inizializzava i moduli `FeatureAlignmentModule` in modo *lazy*, cioè solo al primo `forward`. Il problema è che PyTorch registra i parametri dell'optimizer prima che il training inizi: i pesi di `offset_conv` e `deform_conv` non venivano mai visti da AdamW, che quindi non li aggiornava. In pratica, la `offset_conv` generava offset casuali e costanti, e la backbone IR riceveva in ingresso un rumore spaziale fisso e incorreggibile. L'unica cosa che il modello poteva fare per minimizzare la loss era imparare a ignorare completamente il ramo termico, il che è esattamente quello che ha fatto: la mAP IR-only è crollata a **0.0054**. Il lato positivo, inaspettato, è che forzare il modello a lavorare quasi solo sull'RGB ha prodotto un'ottima performance di fusione (**0.4286**).

Fase 2: Correzione con Eager Initialization (Modello B)

In questo modello i moduli FAM vengono istanziati nel costruttore, passando i canali corretti via `config.encoder_in_channels`. In questo modo i pesi sono registrati correttamente e l'optimizer li aggiorna fin dalla prima iterazione. Il risultato è che la robustezza IR-only è tornata a valori ragionevoli (**0.2629**), ma la fusione è calata a **0.3960**. L'ipotesi è che, potendo ora allinearsi correttamente, il modello tenda ad appoggiarsi troppo sulle feature termiche (più "facili" per localizzare le sagome), a discapito dell'apprendimento visivo.

Fase 3: FAM congelato (Modello C)

Per capire se il beneficio del Modello A fosse legato al rumore casuale in sé o al fatto che il FAM non si aggiornasse, è stato testato un FAM con pesi congelati. In questo caso il gradiente riesce comunque a fluire all'indietro attraverso il modulo, quindi la backbone IR si adatta parzialmente, arrivando a **0.2250** in IR-only. Le performance di fusione però sono le peggiori del gruppo (**0.3857**): il rumore è costante e la rete impara a "conviverci", senza la spinta regolatrice del caso A.

Fase 4: Spatial Dropout (Modello D)

A questo punto l'idea era di penalizzare il ramo termico in modo controllato, introducendo uno *Spatial Dropout* al 40% prima della fusione additiva (parametro `ir_dropout_rate` nel costruttore). L'effetto sperato era costringere il modello a non fare affidamento esclusivo sull'IR, rafforzando il contributo RGB. In parte ha funzionato: la fusione è risalita a **0.3936** e il VIS-only a **0.2716**. Il limite del dropout è però che azzerava interi canali della feature map, eliminando anche informazione semantica utile (non solo geometrica), il che ha un impatto negativo sulla capacità di riconoscimento termico puro.

Fase 5: Stochastic Spatial Jitter (Modello E)

L'ultima versione formalizza l'idea emersa osservando il Modello A: invece di aggiungere rumore casuale fisso agli offset (come accadeva con il bug), viene aggiunto durante il training un rumore Gaussiano $\mathcal{N}(0, \sigma^2)$ agli offset calcolati dalla `offset_conv`. L'effetto è un tremolio spaziale controllato che rende le coordinate termiche deliberatamente inaffidabili, costringendo il modello a appoggiarsi sull'RGB per la localizzazione precisa, ma senza toccare il contenuto semantico delle feature termiche. Il risultato è il migliore in assoluto per la fusione: **0.4381**.

SSJ e Modal Dropout: perché servono entrambi

Un esperimento aggiuntivo ha applicato l'SSJ senza modal dropout, addestrando cioè il modello sempre in modalità fusione. Le performance sono calate sia in fusione (**0.3890**) che in IR-only (praticamente 0): il modello ha semplicemente smesso di usare il ramo termico, appoggiandosi interamente all'RGB. La conclusione è che l'SSJ da solo non basta: serve il modal dropout (configurazione 20% IR, 20% RGB, 60% Fusion) per obbligare la rete a mantenere funzionante anche il ramo termico, privandola periodicamente dell'RGB durante il training.

Analisi Comparativa

La Tabella 11 sintetizza i risultati di tutti i modelli testati con diverse soglie di confidenza (*Thr*).

Non solo l'SSJ ottiene il miglior risultato in fusion, ma evita del tutto i disastri unimodali di annullamento dell'originario bug Lazy, producendo per il solo ramo termico una `map@50` di circa **0.1818**.

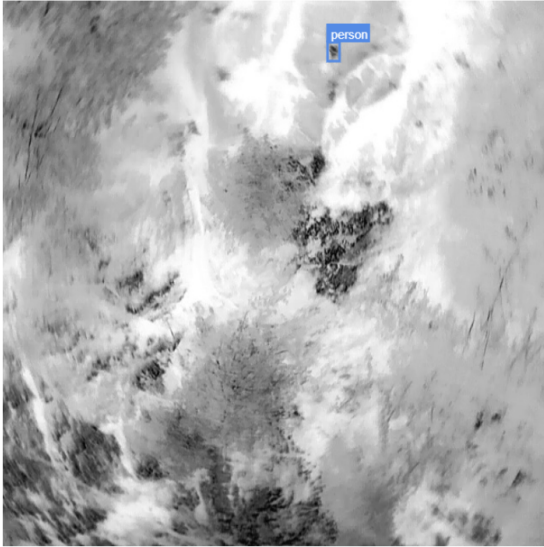
Table 11: Evoluzione architetturale del FAM e l’impatto dello Spatial Jitter (mAP@50).

Dataset	Thr	Modello A (Lazy Bug)	Modello B (Eager FAM)	Modello C (Freeze FAM)	Modello D (Spatial Dropout)	Modello E (Spatial Jitter)
$D_{\text{RGB-IR}}$	0.01	0.4286	0.3960	0.3857	0.3936	0.4381
	0.10	0.4277	0.3944	0.3840	0.3774	0.4363
D_{IRo}	0.01	0.0054	0.2629	0.2250	0.2248	0.1818
	0.10	0.0044	0.2627	0.2206	0.2167	0.1780
D_{VISo}	0.01	0.3000	0.2620	0.2517	0.2716	0.2697
	0.10	0.2987	0.2613	0.2515	0.2674	0.2695

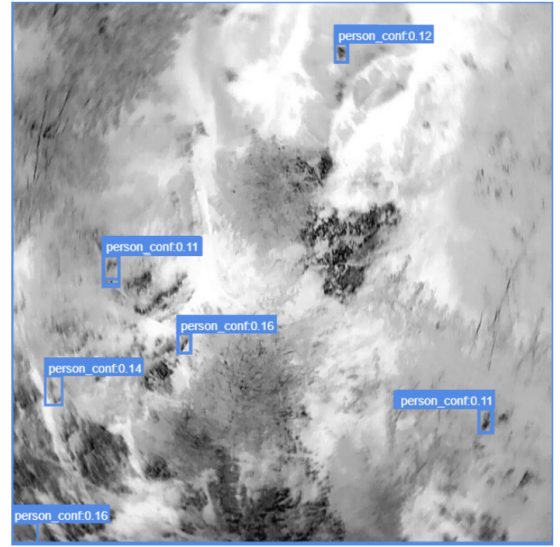
Analisi Qualitativa: Precision vs Recall sul Dataset IR-only

Nonostante la netta differenza in termini di mAP@50 tra il Modello A (0.0044) e il Modello E (0.1780), un’analisi qualitativa delle predizioni rivela un comportamento opposto, in cui il modello A adotta una strategia diversa ma non per forza peggiore.

Come mostrato in Figura 2, su immagini dove il segnale termico è debole o ambiguo, il Modello A preferisce non generare alcuna predizione, mentre il Modello E genera multiple bounding box, trovando il target corretto tra diversi falsi positivi. Su immagini invece dove il segnale è chiaro (Figura 3), il Modello A individua il target con confidenza più alta (0.19 vs 0.14) senza generare i falsi positivi prodotti dal Modello E.



(a) Modello A: nessuna predizione generata.

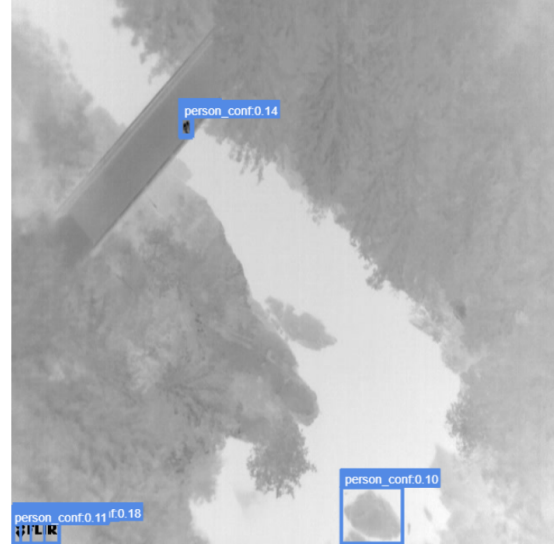


(b) Modello E: target corretto trovato tra diversi falsi positivi.

Figure 2: Confronto su immagine IR con segnale termico debole.



(a) Modello A: target individuato correttamente (conf 0.19), nessun falso positivo.



(b) Modello E: target individuato (conf 0.14) ma con falso positivo aggiuntivo in basso a destra.

Figure 3: Confronto su immagine IR con segnale termico chiaro.

Questa differenza comportamentale è direttamente riconducibile al lazy bug: non avendo mai aggiornato i pesi del FAM sul ramo IR, il modello ha sviluppato un detector estremamente conservativo sull'IR, che genera predizioni solo in presenza di segnali molto forti. Il risultato è alta precision ma recall quasi nullo, infatti su circa il 38% delle immagini del dataset IR-only non viene generata nessuna predizione, e la mAP@50, penalizza severamente questo comportamento.

In un contesto operativo reale, questo trade-off avrebbe implicazioni pratiche importanti, si potrebbe preferire il comportamento del Modello A (poche segnalazioni ma affidabili) rispetto a quello del Modello E (alta copertura ma con molti falsi allarmi).

3 Integrazione di DINO (DETR with Improved DeNoising Anchor Boxes)

Dopo aver consolidato l'architettura di base con Deformable DETR e introdotto i moduli di fusione non lineare e l'allineamento spaziale (FAM/SSJ), il passo successivo è stato integrare i miglioramenti introdotti da **DINO**. DINO ottimizza la fase di addestramento e l'affinamento predittivo del modello, mantenendo inalterata la backbone di fusione multimodale descritta nel capitolo precedente (la classe eredita infatti direttamente da `DeformableDetrFusionFAMForObjectDetection`, incluso il blocco `channel_fusion` $\text{Conv}_{1\times 1} + \text{GroupNorm} + \text{ReLU}$).

L'implementazione (fornita nei moduli `dino_fusion.py` e `dino_cdn.py`) espande il modello di base introducendo due componenti chiave di DINO: il Contrastive DeNoising (CDN) e il Look-Forward-Twice (LFT).

3.1 Contrastive DeNoising (CDN)

Il Contrastive DeNoising Training mira a stabilizzare l'accoppiamento bipartito (*bipartite matching*) tipico di DETR, accelerando la convergenza e riducendo le ambiguità sulle predizioni duplicate. A tale scopo, durante l'addestramento, l'algoritmo inietta direttamente nel decoder delle query pre-impostate (*denoising slots*) derivate dalle annotazioni di Ground Truth (GT), aggiungendo del rumore controllato descritto in `dino_cdn.py`.

Nel nostro framework, il processo di generazione delle query CDN prevede:

- **Generazione di Istanze Positive e Negative:** Vengono costruite copie multiple (divise in gruppi) per ogni bersaglio reale. Nelle controparti *positive*, le etichette di classe e le coordinate spaziali del *bounding box* subiscono perturbazioni stocastiche controllate (modulabili tramite `label_noise_prob` e `box_noise_scale`). Le controparti *negative* subiscono una traslazione spaziale deliberatamente maggiore e sono supervisionate verso la classe "sfondo" (no-object).
- **Gestione dell'Attenzione (Masking):** Una matrice di attention mask impedisce qualsiasi comunicazione tra le query di denoising e le canoniche *matching queries*, e vincola la reciproca attenzione delle query di denoising al solo gruppo di appartenenza.
- **Loss Ausiliaria Dedicata:** L'errore dei token CDN è calcolato separatamente e aggiunto alla loss totale (pesato dal `cdn_loss_coef`). Comprende L1 + GIoU per il ramo regressivo spaziale e una focal loss per la confidenza di classe.

3.2 Look-Forward-Twice (LFT) Decoder

Il secondo potenziamento architetturale introdotto con DINO interviene sulla progettazione del *decoder* deformabile. Nella formulazione originale del Deformable DETR, le coordinate dei box aggiornate (*box refinement*) subiscono l'operazione di `.detach()` nel passaggio da un layer decodificatore al successivo: questo interrompe il flusso del gradiente a ritroso lungo le *reference points*.

In `dino_fusion.py` è stato istanziato uno specifico `LFTDecoder` che rimuove questa operazione, permettendo al gradiente di fluire interamente lungo l'intera cascata di raffinamento progressivo del bounding box, offrendo a ciascun layer una "seconda occasione" (*look forward twice*) di correggere la propria stima sulla base dell'errore osservato nei layer successivi.

3.3 Risultati

Trattandosi della stessa famiglia architetturale di Deformable DETR, tutte e tre le configurazioni DINO testate ereditano lo stesso protocollo a 5 seed e la stessa scelta di riportare mediana e range (§1.4). La Tabella 12 riporta i risultati a soglia di confidenza 0.01.

Table 12: Risultati DINO (CDN + LFT) nelle tre configurazioni di fusione, confidence threshold 0.01. Mediana [range] su 5 run con seed differenti. RT-DETR base riportato come riferimento (singola run, attenzione standard deterministica).

Configurazione	D_{VISo}	D_{IRo}	$D_{\text{RGB-IR}}$
DINO (CDN + LFT)	0.177 [0.139–0.217]	0.116 [0.077–0.146]	0.251 [0.229–0.302]
+ FAM	0.141 [0.117–0.175]	0.113 [0.096–0.154]	0.265 [0.209–0.299]
+ FAM + SSJ	0.149 [0.075–0.193]	0.071 [0.062–0.132]	0.243 [0.105–0.277]
RT-DETR base (singola run)	0.246	0.252	0.357

Il pattern osservato per la famiglia Deformable DETR si replica con notevole coerenza anche su DINO: il FAM produce un miglioramento sulla mediana rispetto alla baseline (0.265 contro 0.251 in $D_{\text{RGB-IR}}$), mentre l’aggiunta dello SSJ riporta la mediana verso il basso (0.243) e produce il range più ampio dell’intero studio (0.105–0.277 in $D_{\text{RGB-IR}}$). La replicazione del medesimo pattern su due architetture indipendenti (Deformable DETR e DINO) rafforza l’ipotesi che l’interazione negativa tra SSJ e non-determinismo del kernel deformabile non sia un artefatto specifico di una singola configurazione, ma un fenomeno sistematico legato all’attenzione deformabile stessa. Nessuna delle tre configurazioni DINO supera RT-DETR base (0.357); il confronto con la famiglia Deformable DETR senza CDN/LFT (Tabella 10) non evidenzia vantaggi sistematici attribuibili a CDN e LFT su questo dataset.

Si nota che il risultato della singola run esplorativa di DINO + FAM + SSJ condotta prima dell’adozione del protocollo a 5 seed ($\text{mAP@50} = 0.2427$) coincide quasi esattamente con la mediana ora calcolata (0.243): una coincidenza che tuttavia non avrebbe potuto essere verificata senza l’esecuzione delle run aggiuntive, confermando a posteriori la necessità del protocollo multi-seed per questa famiglia di architetture.

Riepilogo dei Risultati

Table 13: Riepilogo finale delle prestazioni (mAP@50) dei modelli sviluppati a partire da RT-DETR. Tutti i risultati sono calcolati con soglia di confidenza 0.01. I punteggi migliori per ogni metrica sono evidenziati in grassetto. [†] Mediana su 5 run con seed differenti; range completo in Tabella 10 (famiglia Deformable DETR) e Tabella 12 (DINO).

Modello	Train Dataset	Test Dataset		
		$D_{\text{RGB-IR}}$	D_{IRo}	D_{VISo}
RT-DETR Base	$D_{\text{RGB-IR}}$	0.357	0.252	0.246
RT-DETR Tiling	$D_{\text{RGB-IR}}$	0.151	0.105	0.050
RT-DETR CMX Puro	$D_{\text{RGB-IR}}$	0.032	0.081	0.018
RT-DETR CMX Ibrido	$D_{\text{RGB-IR}}$	0.145	0.094	0.102
Deformable DETR (baseline) [†]	$D_{\text{RGB-IR}}$	0.249	0.115	0.157
Deformable DETR + FAM [†]	$D_{\text{RGB-IR}}$	0.294	0.131	0.184
Deformable DETR + FAM + SSJ [†]	$D_{\text{RGB-IR}}$	0.263	0.088	0.142
RT-DETR FAM (A - Lazy init)	$D_{\text{RGB-IR}}$	0.429	0.005	0.300
RT-DETR FAM (B - Eager FAM)	$D_{\text{RGB-IR}}$	0.396	0.263	0.262
RT-DETR FAM (C - Freeze FAM)	$D_{\text{RGB-IR}}$	0.386	0.225	0.252
RT-DETR FAM (D - Spatial Dropout)	$D_{\text{RGB-IR}}$	0.394	0.225	0.272
RT-DETR FAM (E - Spatial Jitter)	$D_{\text{RGB-IR}}$	0.438	0.182	0.270
DINO (CDN + LFT) [†]	$D_{\text{RGB-IR}}$	0.251	0.116	0.177
DINO + FAM [†]	$D_{\text{RGB-IR}}$	0.265	0.113	0.141
DINO + FAM + SSJ [†]	$D_{\text{RGB-IR}}$	0.243	0.071	0.149

Analisi Comparativa Finale

La campagna sperimentale condotta su diverse architetture (DETR, Deformable DETR, RT-DETR, DINO) e strategie di fusione (Additive, CMX, FAM) permette di fare un quadro chiaro delle sfide legate al rilevamento multimodale in scenari di ricerca e soccorso.

Dall'analisi dei risultati emergono le seguenti evidenze principali:

- **Efficienza vs. Complessità:** L'architettura **RT-DETR** si è rivelata la scelta strutturale più solida per applicazioni real-time. Rispetto a DETR originale (lento) e Deformable DETR (computazionalmente oneroso se non ottimizzato con *Channel Concatenation*), RT-DETR offre il miglior bilanciamento. I tentativi di aumentare la complessità del modulo di fusione tramite framework pesanti come CMX (Zhang et al.) sono falliti a causa dell'inerzia di addestramento su dataset di dimensioni contenute, portando a fenomeni di interferenza distruttiva.
- **Il Dilemma dell'Allineamento (FAM) e la Soluzione SSJ:** L'introduzione del **Feature Alignment Module (FAM)** ha rappresentato una svolta prestazionale su RT-DETR, portandolo a superare agilmente il modello base (0.357). Inizialmente ciò ha introdotto una forte dipendenza dalla modalità guida, al punto da far crollare la robustezza in *single-modality* (IR-only a 0.005 mAP). L'introduzione dello **Stochastic Spatial Jitter (SSJ)** ha risolto definitivamente questo dilemma su RT-DETR: regolarizzando spazialmente le feature, si è ristabilita un'ottima indipendenza dell'IR (0.181 mAP) spingendo al contempo le prestazioni di fusione al picco di **0.438 mAP**. Sulla famiglia Deformable DETR/DINO, lo stesso schema FAM + SSJ non riproduce questo effetto stabilizzante (cfr. evidenza successiva).
- **Affidabilità Statistica e Instabilità dell'Attenzione Deformabile:** L'intera famiglia di modelli basata su Deformable DETR (baseline, FAM, FAM + SSJ, DINO) presenta una variabilità run-to-run, a parità di seed, assente in RT-DETR, riconducibile al kernel CUDA non deterministico della Multi-Scale Deformable Attention e amplificata dal matcher di Hungarian (cfr. §1.4). Su 5 seed per configurazione, nessuna delle varianti testate è statisticamente distinguibile dalle altre: il FAM mostra la tendenza più promettente, ma confermarla richiederebbe tra 13 e 48 run per configurazione, a seconda della metrica di confronto adottata. Il FAM + SSJ, in particolare, mostra la varianza più ampia osservata in tutto lo studio, suggerendo un'interazione negativa tra il rumore iniettato dallo SSJ e il non-determinismo del kernel.
- **Evoluzione Architetture (DINO):** L'implementazione di DINO (CDN + LFT) nelle tre varianti di fusione (baseline, + FAM, + FAM + SSJ) produce mediane di $D_{\text{RGB-IR}}$ pari rispettivamente a 0.251, 0.265 e 0.243 — dello stesso ordine di grandezza delle corrispondenti configurazioni Deformable DETR senza CDN/LFT (0.249, 0.294, 0.263) e in tutti i casi inferiori al singolo run disponibile per RT-DETR base (0.357). Il pattern FAM $\uparrow$ / FAM+SSJ $\downarrow$ con range in esplosione si ripete identico su entrambe le architetture, rafforzando l'ipotesi di un'interazione sistematica tra SSJ e non-determinismo del kernel deformabile piuttosto che un artefatto specifico di una singola configurazione. Non vi è quindi evidenza che CDN e LFT migliorino le prestazioni rispetto alla baseline defDETR o a RT-DETR su questo dataset.
- **Strategia di Input:** Il modello addestrato su immagini integrali (640×640) ha surclassato costantemente la strategia di *Tiling*. La perdita di contesto globale e la ridotta frequenza di campionamento dei soggetti nei ritagli hanno reso il tiling inefficace, confermando che la sfida principale per questo task è data dal disallineamento geometrico dei sensori e non dalle piccole dimensioni dei soggetti.

Conclusioni e Prospettive Future

Alla luce dei risultati ottenuti possiamo evidenziare due soluzioni ottimali in base ai vincoli operativi:

1. **Robustezza Operativa Estrema:** Se la priorità vitale è garantire il rilevamento anche in caso di oscuramento o malfunzionamento critico del sensore visivo (es. fumo denso o buio totale), l'architettura **RT-DETR con Additive Feature Fusion pura o Late Fusion** rappresenta la scelta più invariante. Garantisce prestazioni solide e bilanciate sia in modalità combinata che singola, pur perdendo i picchi di precisione geometrica offerti dai moduli di allineamento.
2. **Massima Precisione Multimodale:** L'architettura **RT-DETR + FAM + SSJ** rappresenta allo stato attuale la configurazione definitiva. La capacità del modulo FAM di correggere i disallineamenti sfruttando la risoluzione RGB, unita alla regolarizzazione del Jitter stocastico, permette di massimizzare il punteggio mAP portandolo al record di **0.438**, senza sacrificare la stabilità operativa in fase addestrativa.

Va sottolineato che, allo stato attuale, nessuna delle architetture basate su Deformable DETR (baseline, FAM, FAM + SSJ, DINO) eguaglia le prestazioni di RT-DETR in fusione; inoltre, l'elevata variabilità run-to-run di queste architetture (cfr. §1.4) impone cautela nell'interpretare le differenze osservate tra le loro varianti come miglioramenti reali piuttosto che rumore statistico.

Sviluppi Futuri

La naturale evoluzione di questo lavoro richiede, in primo luogo, di risolvere il problema della potenza statistica prima di trarre conclusioni architettureali definitive sulla famiglia Deformable DETR/DINO: l'analisi di potenza condotta (cfr. §1.4) indica che sarebbero necessarie circa 13 run per configurazione per confermare il guadagno mediano osservato con il FAM, e fino a 48 per confermarne il guadagno medio. La coerenza del pattern FAM/SSJ osservata su sei configurazioni indipendenti (tre per defDETR, tre per DINO) costituisce un'evidenza incoraggiante, ma non sufficiente a conclusioni statisticamente consolidate con le risorse computazionali disponibili.